Matière : Dev. App. Rép.
Niveau d'études : L3 Info



Enseignante *: Malek Ltaief*
Année universitaire : 2026-2027

Suite TP1 : Serveurs TCP multi-clients en JAVA

Objectifs

À l'issue de ces deux exercices, vous serez capable de :

— Programmer un serveur TCP **itératif** qui traite les clients l'un après l'autre ;

— Programmer un serveur TCP **concurrent** en utilisant les threads.

Contexte

Le club **Pattes Solidaires** est lancé. Le samedi, plusieurs bénévoles sont sur le terrain en même temps et se connectent au **serveur central** du club. Le serveur doit donc gérer **plusieurs clients**.

Exercice 4 : Serveur multi-clients itératif

Le samedi matin, plusieurs bénévoles arrivent en même temps au point de nourrissage et se connectent au serveur central pour **signaler leur présence** et recevoir leur zone d'intervention. Le serveur est unique et traite **un bénévole à la fois**.

Travail demandé

1. **Serveur.** Développer un serveur TCP qui écoute sur le port `9977` et qui, plusieurs clients itérativement. Pour chaque client, le serveur :
   — accepte une connexion ;
   — lit une ligne `prenom;zone` (par exemple `Sarra;JARDIN`) ;
   — répond par une ligne : `"Bienvenue {prenom}, vous êtes le bénévole n°{n} de la journée. Zone : {zone}."`, où `n` est un compteur incrémenté à chaque bénévole ;
2. **Client.** Développer le client (bénévole) : il lit au clavier son prénom et sa zone, envoie `prenom;zone`, affiche la réponse du serveur, puis se déconnecte.

Expérimentation et questions

Lancer le serveur, puis lancer **trois clients** dans trois terminaux : le client 1 se connecte mais ne saisit rien pendant quelques secondes, les clients 2 et 3 saisissent leurs informations aussitôt.

a) Dans quel ordre les clients sont-ils servis ? Que constate-t-on côté clients 2 et 3 tant que le client 1 n'a rien envoyé ?

b) Pourquoi `ss.close()` ne doit-il pas se trouver **à l'intérieur** de la boucle ?

c) Un client se connecte mais **n'envoie rien**. Que devient le serveur ? Quelle limite de l'approche itérative cela illustre-t-il ?

Exercice 5 : Serveur multi-clients concurrent

Le problème de l'exercice 4 est réel : un bénévole qui traîne bloque tous les autres. Le serveur doit maintenant servir **plusieurs bénévoles en même temps**, chacun enregistrant ses fiches animaux dans un **registre commun**. On réutilise les classes `FicheAnimal` et `AccuseReception` de l'exercice 3.

Travail demandé

1. Implémenter la classe `ServiceFiche`. Cette méthode permet de :
   — lire un entier `n` (nombre de fiches à enregistrer), puis `n` objets `FicheAnimal` ;
   — pour chaque fiche, appelle `estFicheValide()` ; si la fiche est valide, l'ajoute au **registre partagé** et renvoie un `AccuseReception` de statut `"OK"` avec son numéro d'ordre ; sinon, renvoie un `AccuseReception` de statut `"REJETE"` avec un message d'erreur ;
2. Implémenter le serveur `ServeurRegistreMT` : une boucle infinie qui accepte une connexion, crée un `ServiceFiche`, le lance dans un **nouveau thread**.
3. Réutiliser le client de l'exercice 3, adapté pour envoyer `n` fiches saisies au clavier et afficher chaque accusé de réception.

```
public class ServiceFiche implements Runnable {
  private Socket s;
  private static ArrayList<FicheAnimal> registre = new ArrayList<FicheAnimal>();
      // partagé
  private static int dernierNumero = 0;    // partagé

  public ServiceFiche(Socket s) { this.s = s; }

  public void run() {
    try {
      // TODO
        } catch (Exception e) { e.printStackTrace(); }
  }
}
```

Expérimentation et questions

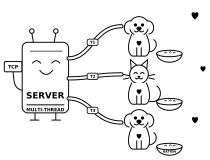
Lancer le serveur et trois clients, en refaisant le même scénario qu'à l'exercice 4 (le client 1 tarde à saisir ses informations).

a) Les clients 2 et 3 sont-ils encore bloqués par le client 1 ? Comparer avec l'exercice 4 et expliquer la différence.

b) Calculer d'abord `numeroOrdre` avec `dernierNumero++` **sans précaution**. Avec beaucoup de clients simultanés, peut-on obtenir des numéros dupliqués ? Expliquer pourquoi.

c) Corriger le problème avec le mot-clé `synchronized`. Quelle partie du code doit être protégée, et laquelle ne doit **pas** l'être ? Pourquoi ?

d) Pourquoi le serveur ne peut-il pas utiliser `FicheAnimal.getCompteurFiches()` comme numéro d'ordre ?



One server, one volunteer at a time : patience is a virtue, but queues are tiring !
Threads let every volunteer in at once : no cat waits at the door !